

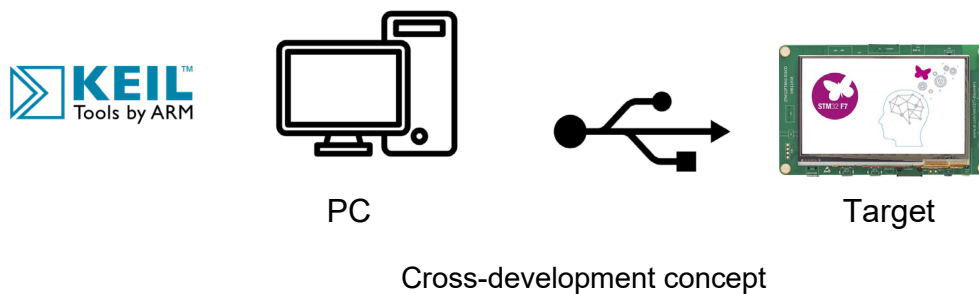


ARM CROSS DEVELOPMENT (INTRO)

1 INTRODUCTION

This laboratory aims to introduce you to the cross-development tools used in this course. Using programs written in assembler, the various steps involved in cross-development will be practiced.

The figure below illustrates the cross-development concept. The host computer (PC) is equipped with the Keil uVision development environment (IDE). This enables code to be written, compiled, and downloaded onto a target. Using the Keil uVision IDE, a program running on the target can also be debugged.



ARM Keil uVision version 5 will be the development environment (IDE) for this course. The ARM Compiler (armcc) will be used.

2 DEV. KIT STM32F746G-DISCO

The development board used is the STM32F746G-DISCO supplied by ST-Microelectronic, with a custom extension board. The microcontroller used is an ARM Cortex M7 from ST.

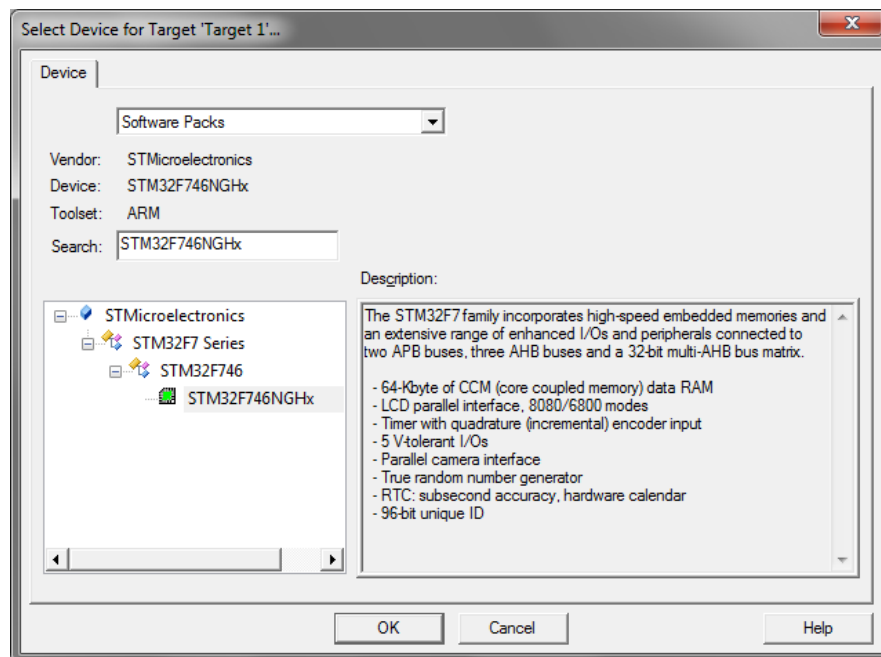
- Dev kit: 32F746GDISCOVERY
- MCU: Arm® Cortex®-M7-core-based STM32F746NG (STM32F746NGH6 order code)
- ARMv7-M architecture, Thumb2 instruction set (16 bits compressed or 32 bits).



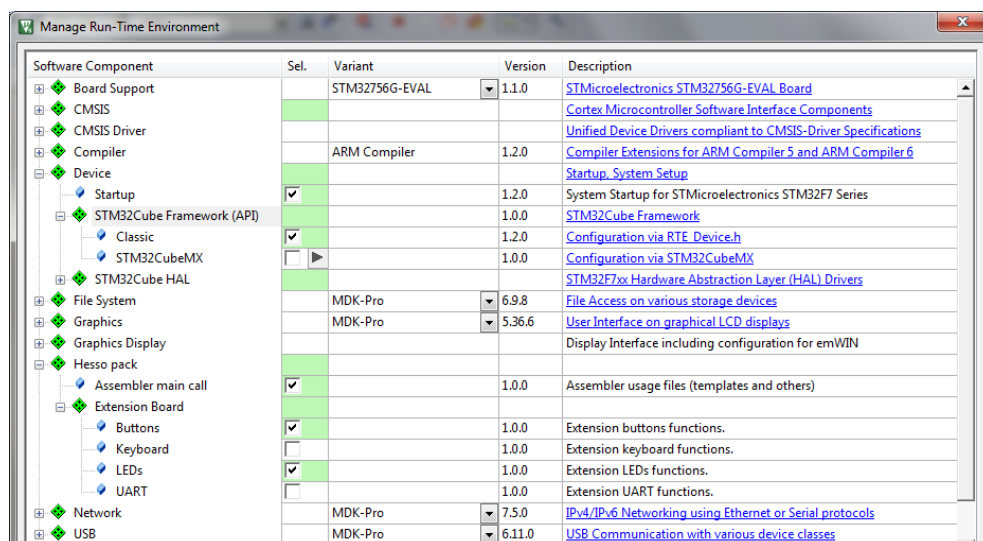
3 FIRST KEIL PROJECT

No files are provided for this lab. Please follow the procedure below to create a new project using Keil uVision.

1. Start Keil uVision IDE. In the "Project" menu, select "New uVision Project...".
2. Select a local folder and path without spaces or special characters.
3. For this course, the processor used is the STM32F746NGH6. Please select the STM32F746NGHx device from the Software Packs.



4. You can now select the components and peripherals you wish to use. This selection can be made graphically, using the screen below (the pack versions may differ).



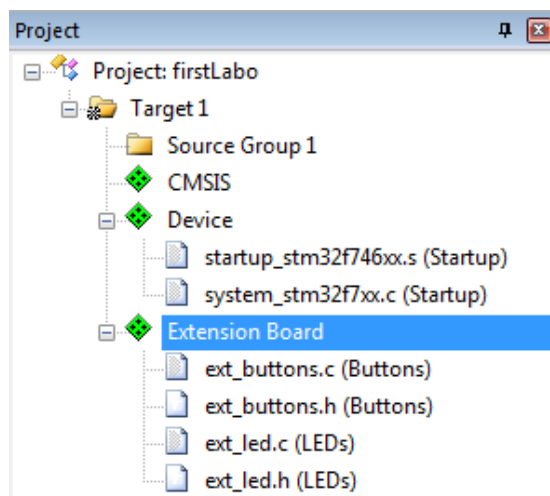


For the first lab, we're going to use the buttons and LEDs from the extension board. These HES-SO Pack components must therefore be selected. These packs provide the necessary functions and APIs.

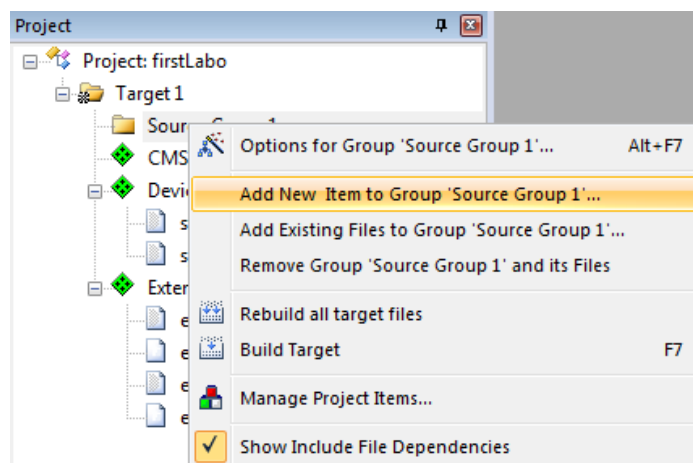
You'll only be writing assembly code for this first lab. To do this, select the Device/Startup component with the classic STM32Cube Framework API, as shown in the figure above.

Finally, resolve all dependencies by pressing the "Resolve" button to activate missing dependencies (if so, orange will appear and everything should be green at the end).

5. A new project has been created and various files copied, according to the following structure:



6. You still need to add your source / "main" file. Use the contextual menu on the "Source Group 1" folder.



You can create empty files or use existing templates.

For this laboratory, create an Asm (assembler) "main" file. The file created will be "main.s". This is a text file, your source code, composed of assembler instructions.



7. Add the following code to the "main.s" file you have created.
Your program's main procedure must be named "asm_main".

```
; Your first ARM assembly program (compiles but does nothing)

AREA    |.text|, CODE, READONLY

EXPORT  asm_main
IMPORT  Ext_Buttons_Init
IMPORT  Ext_Buttons_GetState
IMPORT  Ext_LED_Init

asm_main PROC
    bl    Ext_Buttons_Init
    bl    Ext_LED_Init

loop
    bl    Ext_Buttons_GetState
    b     loop
ENDP

END
```

First ARM assembly program

4 CODE COMPILATION

Use "Project - Build Target" or the F7 key to compile your assembler project.

The result of compilation is visible in the "Build Output" window, as shown below.

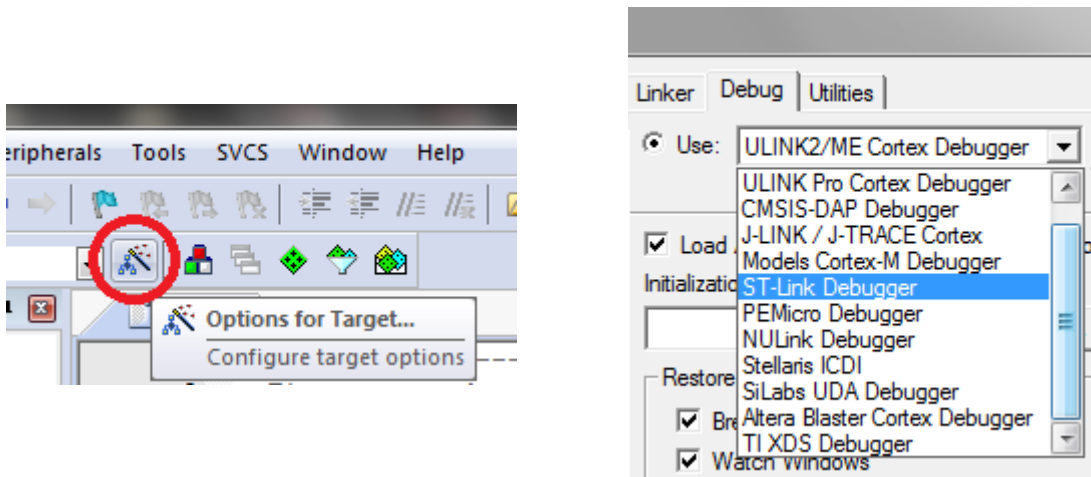
```
Build Output
*** Using Compiler 'V5.06 update 5 (build 528)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Target 1'
assembling main.s...
linking...
Program Size: Code=1764 RO-data=560 RW-data=4 ZI-data=1540
".\Objects\firstLabo.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:00
```



5 DOWNLOAD AND DEBUG

Connect the STM32F746G-Disco board to your PC's USB port.

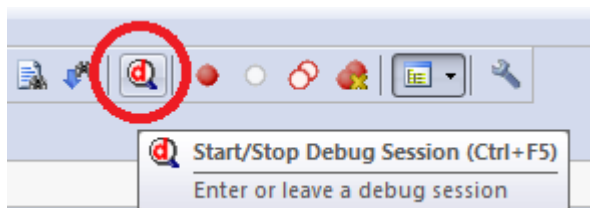
Before downloading the program to the target, we need to define which hardware debugger we're using. Click the "Option for Target" button as shown in the image below.



An ST-Link Debugger is directly integrated on the development board. Select the ST-Link Debugger in the Debug tab.

Then click on the Settings button to check that the card is connected directly. Click on OK to validate and close the settings window.

To download the program to the target, perform the following operation. Click the "Start/Stop Debug Session" button, as shown in the image below. Always compile your program before uploading it to the target. This is not done automatically.



The program is now loaded on target. You can debug your program using breakpoints, register views, see the disassembled code and machine code, look at the memory content, etc.



6 TASK 1

The code provided above compiles correctly but has no effect. Complete the provided assembly code from the previous example so that the LEDs represent the current state of the buttons.

Browse LED and button packs and files to discover the different functions available. Use the available functions to complete the task.

7 TASK 2

Download the code provided on Moodle, add it to your project, and test the program's output. This program should make a "K2000"-style light animation on the LEDs. However, it doesn't work as expected. Fix the errors in the provided program. You should see an animation going left to right and back in the opposite direction.

Once the program is running, answer the following questions:

1. At which memory address is `asm_main` located ?
2. What is the opcode of the instruction `subs r7,r7,#1`
3. What is the opcode of the instruction `mov r6,#0`
4. What is the opcode of the instruction `mov r0,#40000`

8 TASK 3

Modify the current program so that the K2000 animation speed depends on the number of button presses.

Implement your solution in assembly by modifying the code from the previous task.